

Motion planning and robots control

Salih ABDELAZIZ

Maître de Conférences à l'UM2

LIRMM, Département robotique

Montpellier, France

abdelaziz@lirmm.fr

OUTLINE

❖ Motion planning

- Joint space
- Cartesian space

❖ Motion control

- Pose control based only on kinematics
- Pose control based on dynamics (computed torque)

❖ Force control

- Implicit force control
- Explicit force control

OUTLINE

❖ Motion planning

- Joint space
- Cartesian space

❖ Motion control

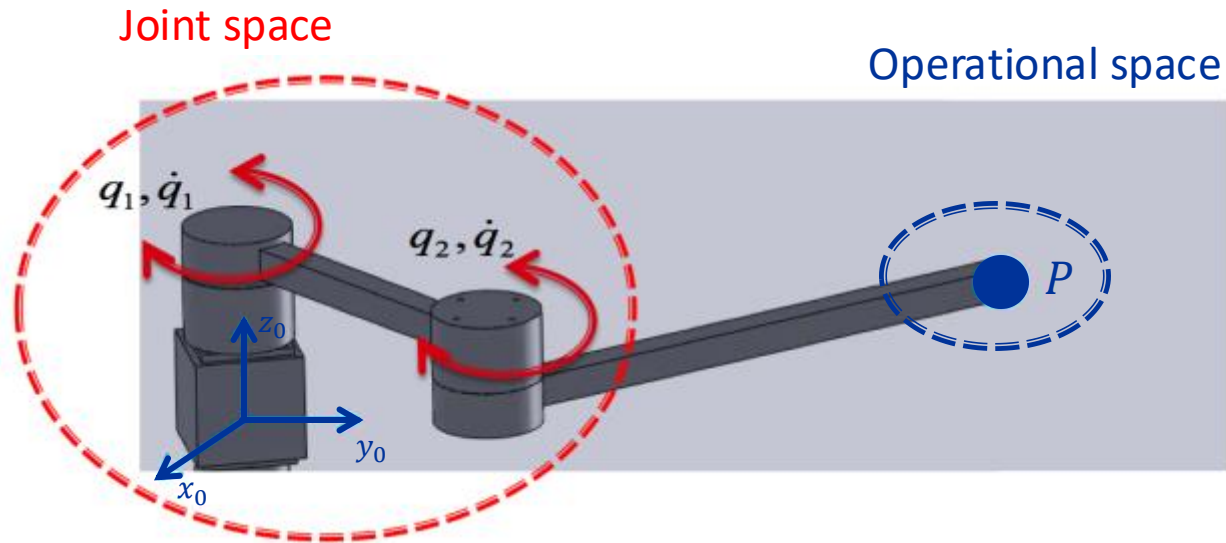
- Pose control based only on kinematics
- Pose control based on dynamics (computed torque)

❖ Force control

- Implicit force control
- Explicit force control

DEFINITIONS

Let be a robot with m degrees of freedom and n joints



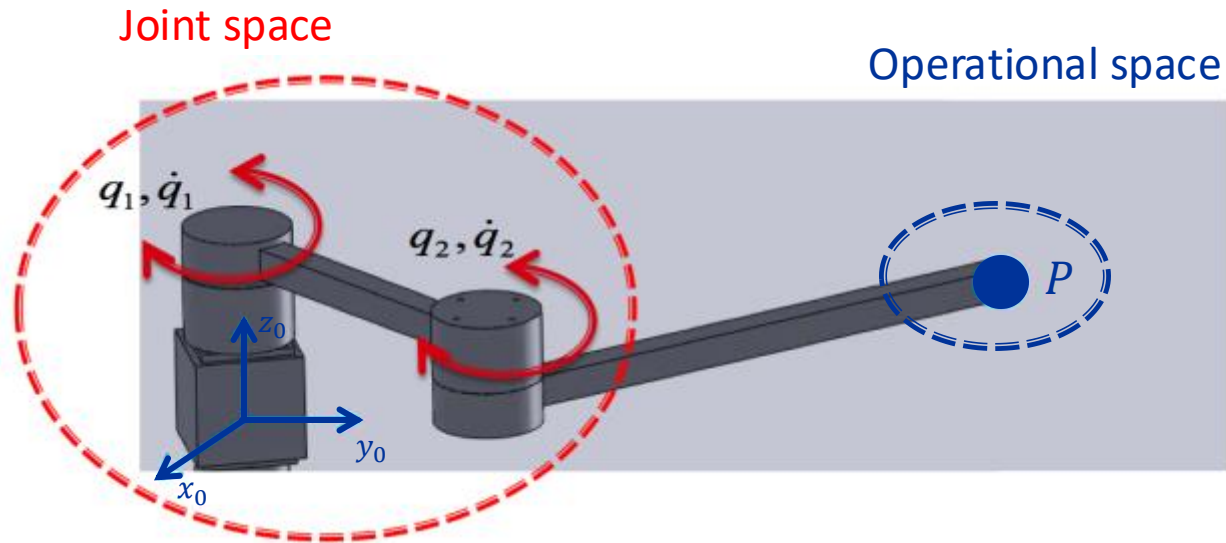
Joint space :

Vector of generalized coordinates: $\mathbf{q} = [q_1 \quad q_2 \quad \dots \quad q_n]^T$

Joint speed vector: $\dot{\mathbf{q}} = [\dot{q}_1 \quad \dot{q}_2 \quad \dots \quad \dot{q}_n]^T$

Vector of joint accelerations: $\ddot{\mathbf{q}} = [\ddot{q}_1 \quad \ddot{q}_2 \quad \dots \quad \ddot{q}_n]^T$

DEFINITIONS



Operational space :

End effector pose: $\mathbf{x} \equiv$ position + orientation de P dans le repère $\mathcal{R}_0(x_0, y_0, z_0)$

Velocity of the end effector : $\dot{\mathbf{x}} = [\dot{x} \quad \dot{y} \quad \dot{z} \quad \omega_x \quad \omega_y \quad \omega_z]^T$

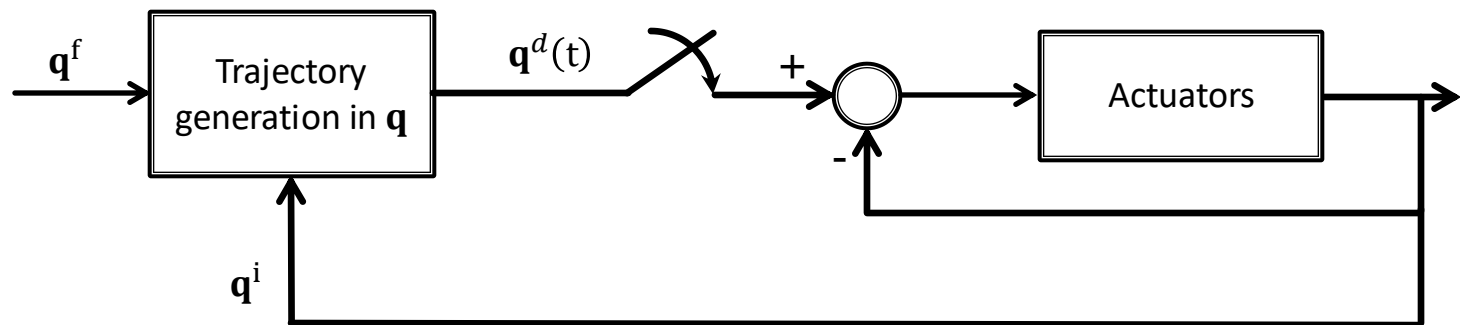
INTRODUCTION TO MOTION PLANNING

- ❖ Motion planning = function for calculating the robot's instructions in the form of successive positions of the end effector or the joints
- ❖
- ❖ Motion class: movement between 2 points
 - via a free trajectory between points. *
 - With a constrained trajectory between the points (e.g. straight line) Δ

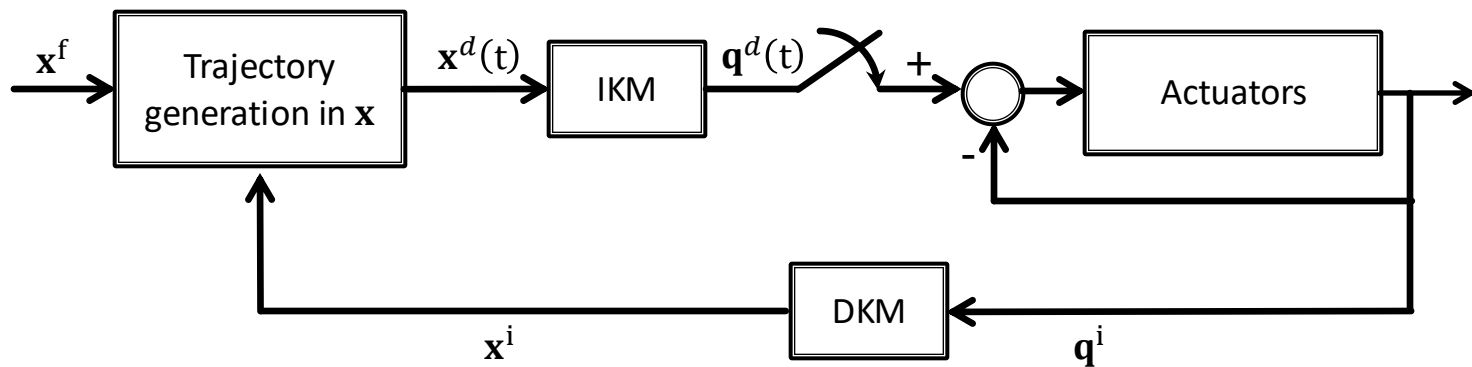
* can be performed in the joint space

Δ can be done directly in the operational space

MOTION PLANNING : JOINT SPACE VS OPERATIONAL SPACE



Motion planning in the joint space



Motion planning in the operational space

MOVEMENT BETWEEN 2 POINTS IN THE JOINT SPACE

❖ Motion between $\mathbf{q}^i$ and $\mathbf{q}^f$ can be described as:

$$\mathbf{q}(t) = \mathbf{q}^i + r(t)\mathbf{D} \quad 0 \leq t \leq t_f$$

$$\text{with} \quad \mathbf{D} = \mathbf{q}^f - \mathbf{q}^i$$

$\mathbf{q}(t) = \mathbf{q}^d(t)$ for simplicity

$\mathbf{q}^i$: vector of the generalized coordinates corresponding to an initial configuration

$\mathbf{q}^f$: vector of the generalized coordinates corresponding to a final configuration

The values at the limits of the function $r(t)$:

$$\begin{cases} r(0) = 0 \\ r(t_f) = 1 \end{cases}$$

MOTION BETWEEN 2 POINTS IN THE JOINT SPACE

5th order polynomial equation:

- ❖ Adapted for high-speed robots or ones carrying large loads
- ❖ ➔ ensure the continuity of acceleration in order to avoid exciting the mechanics
- ❖ Motion of class $\mathcal{C}^2$

Six constraints to satisfy (2 in position, 2 in velocity and 2 for acceleration)

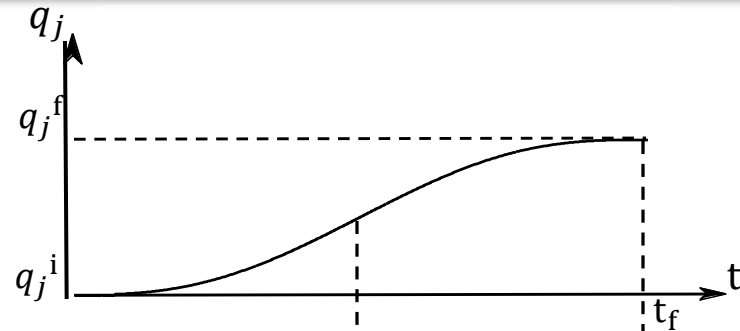
$$\mathbf{a}_0 = \mathbf{q}^i \quad ; \quad \mathbf{a}_1 = \mathbf{a}_2 = \mathbf{0} \quad ; \quad \mathbf{a}_3 = \frac{10}{t_f^3} \mathbf{D} \quad ; \quad \mathbf{a}_4 = \frac{-15}{t_f^4} \mathbf{D} \quad ; \quad \mathbf{a}_5 = \frac{6}{t_f^5} \mathbf{D}$$

$$\mathbf{r}(t) = 10 \left(\frac{t}{t_f} \right)^3 - 15 \left(\frac{t}{t_f} \right)^4 + 6 \left(\frac{t}{t_f} \right)^5$$

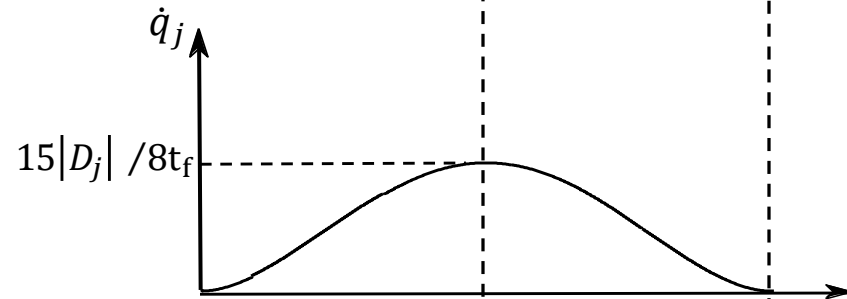
MOTION BETWEEN 2 POINTS IN THE JOINT SPACE

5th order polynomial equation:

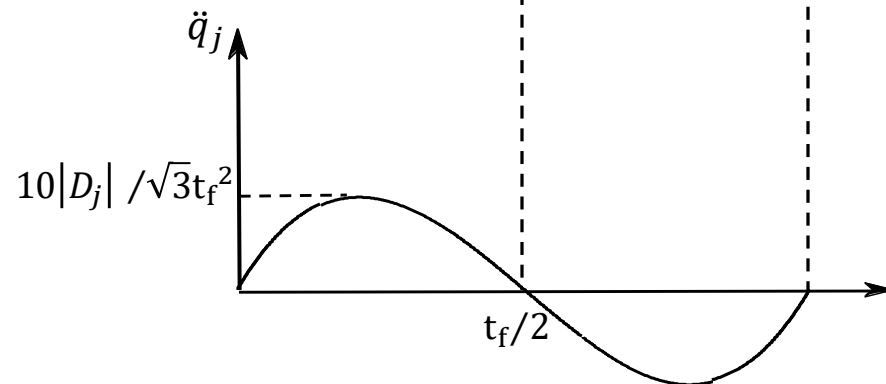
Position



Velocity



Acceleration



MOTION BETWEEN 2 POINTS IN THE JOINT SPACE

The overall minimum time to be applied to all joints is constrained by the slowest joint:

$$t_f = \max(t_{f_1}, t_{f_2}, \dots, t_{f_m})$$

Interpolation function	Minimum time
5th order polynomial equation	$t_{f_j} = \max\left(\frac{15 D_j }{8k_{v_j}}, \sqrt{\frac{10 D_j }{\sqrt{3}k_{a_j}}}\right)$

MOTION BETWEEN 2 POINTS IN THE JOINT SPACE

- ❖ $\mathbf{k}_v = [k_{v_1} \quad \dots \quad k_{v_n}]^T$: vector of maximum joints velocities (motors datasheet)
- ❖ $\mathbf{k}_a = [k_{a_1} \quad \dots \quad k_{a_n}]^T$: vector of maximum joints accelerations (motors datasheet)

MOTION BETWEEN 2 POINTS IN THE JOINT SPACE

Benefits :

- Less online computations (no use of kinematic models)
- Movement not affected by the passage through the singular configurations
- Maximum speed and acceleration constraints directly deduced from the physical limits of the motors
-

Disadvantages :

- The geometry of the end effector trajectory in the operational space not predictable
- Risk of collisions if the robot operates in a cumbersome environment

MOVEMENT BETWEEN 2 POINTS IN OPERATIONAL SPACE

- ❖ We want to generate a trajectory between an initial pose (position and orientation) and a final pose of the end effector. For this, we define :

$\mathbf{x}^i = [x^i \quad y^i \quad z^i]^T$: initial position of the end effector

$\mathbf{x}^f = [x^f \quad y^f \quad z^f]^T$: final position of the end effector

$\mathbf{R}^i$: rotational matrix corresponding to the current (initial) orientation of the end effector with respect to the reference frame

$\mathbf{R}^f$: rotational matrix corresponding to the desired (final) orientation of the end effector with respect to the reference frame

- ❖ let ${}^0\mathbf{T}_e^i$ and ${}^0\mathbf{T}_e^f$ be two matrices describing the initial and final poses of the end effector:

$${}^0\mathbf{T}_e^i = \begin{pmatrix} \mathbf{R}^i & \mathbf{x}^i \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{and} \quad {}^0\mathbf{T}_e^f = \begin{pmatrix} \mathbf{R}^f & \mathbf{x}^f \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

MOVEMENT BETWEEN 2 POINTS IN OPERATIONAL SPACE

- ❖ Motion of the end effector is decomposed into :
 - Translational motion: **straight line**
 - A rotational movement ϑ around an axis **u**
- ❖ The distance D to be covered by the end effector (translational movement) is:

$$D = \|\mathbf{x}^f - \mathbf{x}^i\|_2 = \sqrt{(x^f - x^i)^2 + (y^f - y^i)^2 + (z^f - z^i)^2}$$

- ❖ The calculation of the rotation is done from:

$$\mathbf{R}^i \text{rot}(\mathbf{u}, \vartheta) = \mathbf{R}^f$$

We then try to determine **u** and ϑ

MOVEMENT BETWEEN 2 POINTS IN OPERATIONAL SPACE

- ❖ The solution is calculated by:

$$\mathbf{rot}(\mathbf{u}, \vartheta) = (\mathbf{R}^i)^T \mathbf{R}^f = \begin{pmatrix} x_x & y_x & z_x \\ x_y & y_y & z_y \\ x_z & y_z & z_z \end{pmatrix}$$

$$C\vartheta = \frac{1}{2}(x_x + y_y + z_z - 1)$$

$$S\vartheta = \frac{1}{2}\sqrt{(y_z - z_y)^2 + (z_x - x_z)^2 + (x_y - y_x)^2}$$

- ❖ The vector $\mathbf{u}$ and the angle ϑ are computed as:

$$\vartheta = \text{atan2}(S\vartheta, C\vartheta)$$

$$\mathbf{u} = \begin{pmatrix} u_x \\ u_y \\ u_z \end{pmatrix} = \frac{1}{2 S\vartheta} \begin{pmatrix} y_z - z_y \\ z_x - x_z \\ x_y - y_x \end{pmatrix}$$

MOVEMENT BETWEEN 2 POINTS IN OPERATIONAL SPACE

❖ Finally, the motion of the end effector is described by:

❖

$${}^0\mathbf{T}_E^d(t) = \begin{pmatrix} \mathbf{R}^d(t) & \mathbf{x}^d(t) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Such that:

$$\begin{aligned} \mathbf{x}^d(t) &= \mathbf{x}^i + r(t)(\mathbf{x}^f - \mathbf{x}^i) \\ \mathbf{R}^d(t) &= \mathbf{R}^i \mathbf{rot}(\mathbf{u}, r(t)\vartheta) \end{aligned}$$

with

$$\mathbf{rot}(\mathbf{u}, r(t)\vartheta) = \begin{pmatrix} u_x^2(1 - \cos(r\vartheta)) + \cos(r\vartheta) & u_x u_y(1 - \cos(r\vartheta)) - u_z \sin(r\vartheta) & u_x u_z(1 - \cos(r\vartheta)) + u_y \sin(r\vartheta) \\ u_x u_y(1 - \cos(r\vartheta)) + u_z \sin(r\vartheta) & u_y^2(1 - \cos(r\vartheta)) + \cos(r\vartheta) & u_y u_z(1 - \cos(r\vartheta)) - u_x \sin(r\vartheta) \\ u_x u_z(1 - \cos(r\vartheta)) - u_y \sin(r\vartheta) & u_y u_z(1 - \cos(r\vartheta)) + u_x \sin(r\vartheta) & u_z^2(1 - \cos(r\vartheta)) + \cos(r\vartheta) \end{pmatrix}$$

r has the form:

$$r = r(t) = 10 \left(\frac{t}{t_f}\right)^3 - 15 \left(\frac{t}{t_f}\right)^4 + 6 \left(\frac{t}{t_f}\right)^5$$

MOVEMENT BETWEEN 2 POINTS IN OPERATIONAL SPACE

Benefits:

- Allows to control the geometry of the trajectory

Disadvantage:

- Transformation into joint coordinates of each point in the trajectory
- Possibility of failure if passing through singular positions
- Possibility to fail if the trajectory points are not in the robot's workspace
- Speed and acceleration limits in the operational space vary depending on the robot configuration

OUTLINE

❖ Motion planning

- Joint space
- Cartesian space

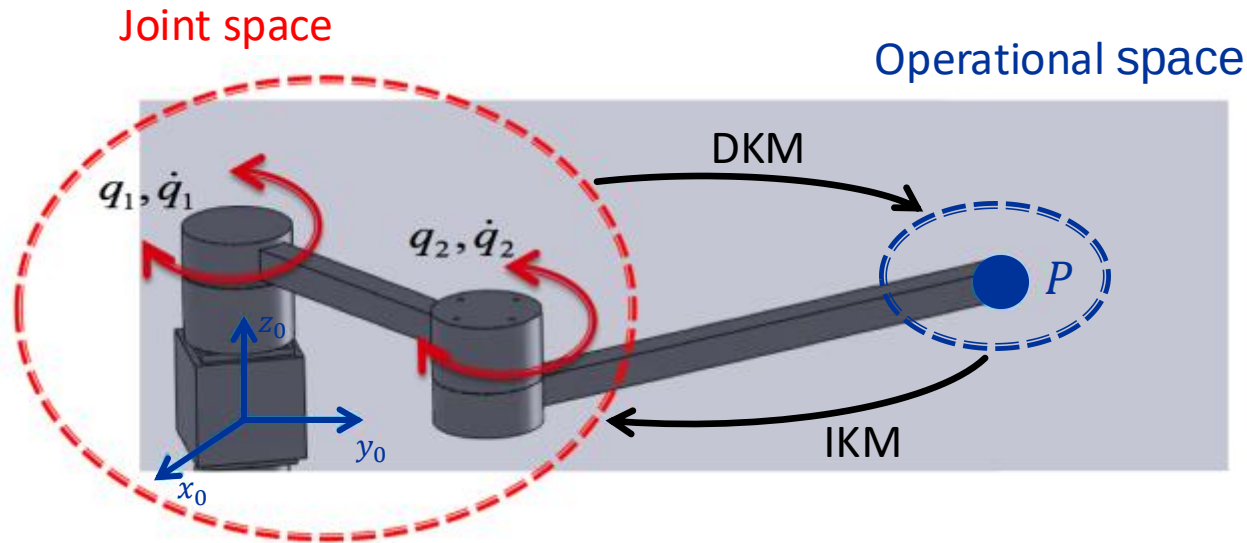
❖ Motion control

- Pose control based only on kinematics
- Pose control based on dynamics (computed torque)

❖ Force control

- Implicit force control
- Explicit force control

KINEMATIC MODELS (DIRECT AND INVERSE)



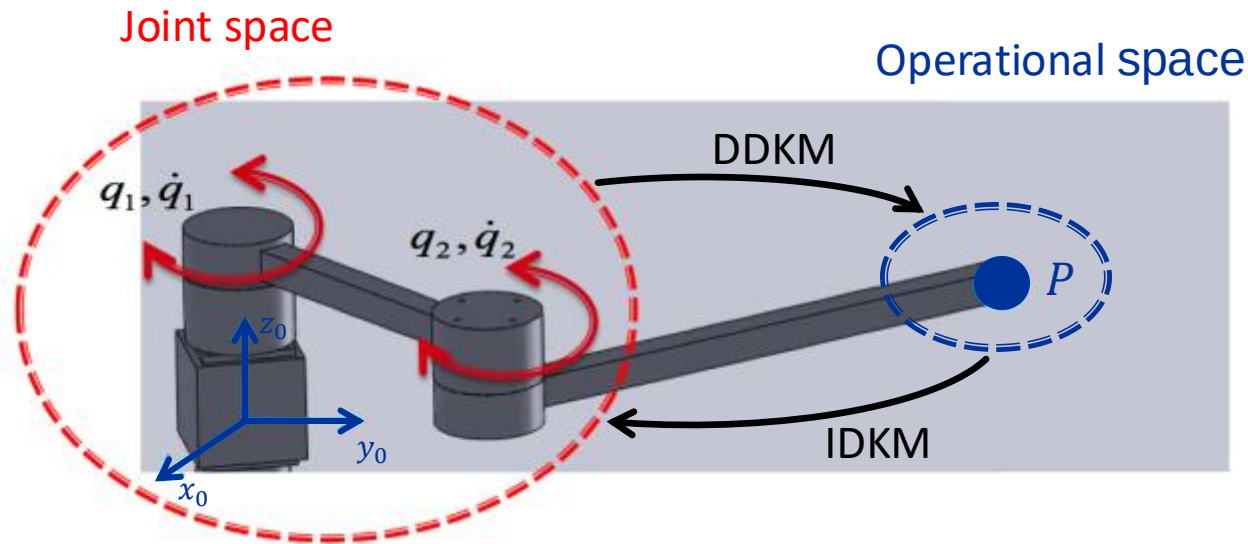
Direct kinematic model (DKM) :

$$\mathbb{x} = \mathbf{g}(\mathbf{q})$$

Inverse kinematic model (IKM) :

$$\mathbf{q} = \mathbf{g}^{-1}(\mathbb{x})$$

DIFFERENTIAL KINEMATIC MODELS (DIRECT AND INVERSE)



Direct differential kinematic model (DDKM) :

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$$

$\mathbf{J}$: Jacobian matrix of the robot of dimension $m \times n$

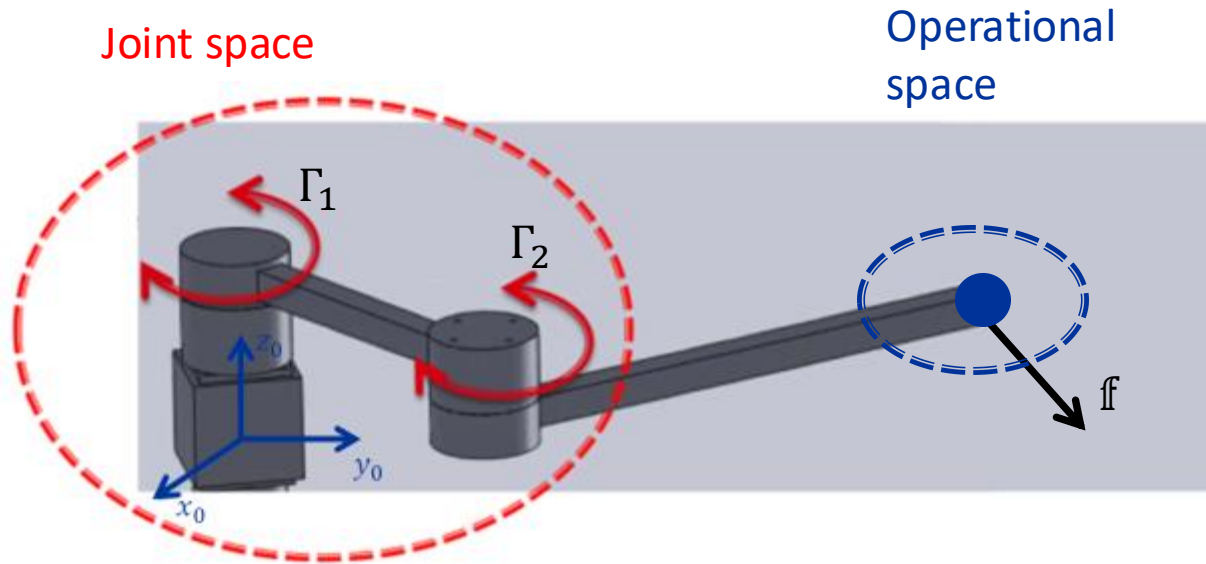
Inverse differential kinematic model (IDKM) :

$$\dot{\mathbf{q}} = \mathbf{J}^\#(\mathbf{q}) \dot{\mathbf{x}}$$

if $m = n \Rightarrow \mathbf{J}^\# = \mathbf{J}^{-1}$

else $\mathbf{J}^\# = \mathbf{J}^+$ (Moore-Penrose inverse or pseudo-inverse)

STATIC MODEL



$\mathbf{\Gamma} = [\Gamma_1 \quad \Gamma_2 \quad \dots \quad \Gamma_n]^T$: vector of joint torques (torques produced by actuators)

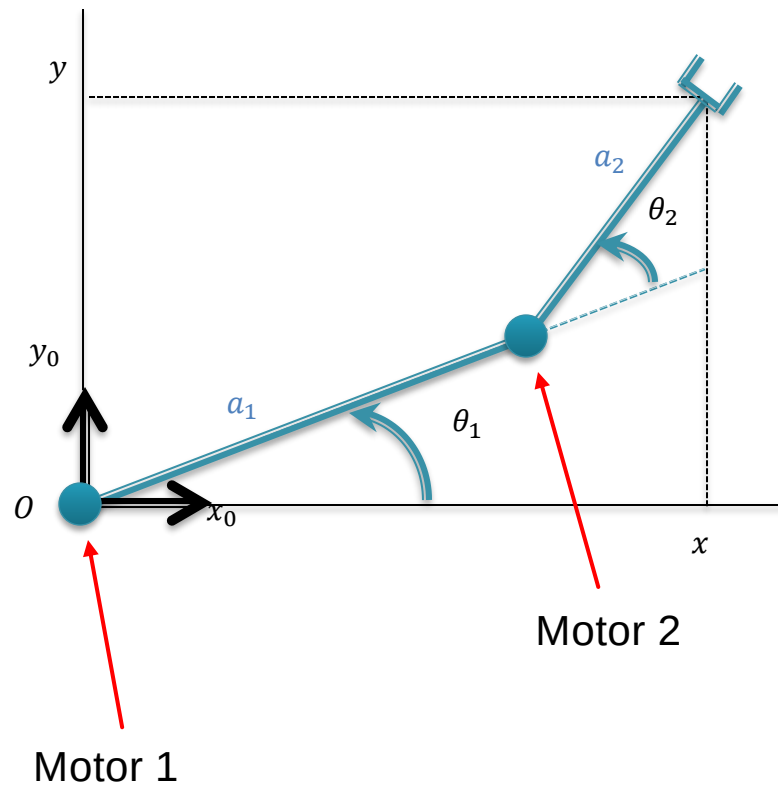
$\mathbf{f} = [f_x \quad f_y \quad f_z \quad m_x \quad m_y \quad m_z]^T$: force and moment applied on the end effector

$$\mathbf{\Gamma} = \mathbf{J}^T(\mathbf{q}) \mathbf{f}$$

$\mathbf{J}^T$: Transpose of the Jacobian matrix of dimension $n \times m$

POSITION CONTROL

❖ 2 DOF planar robot

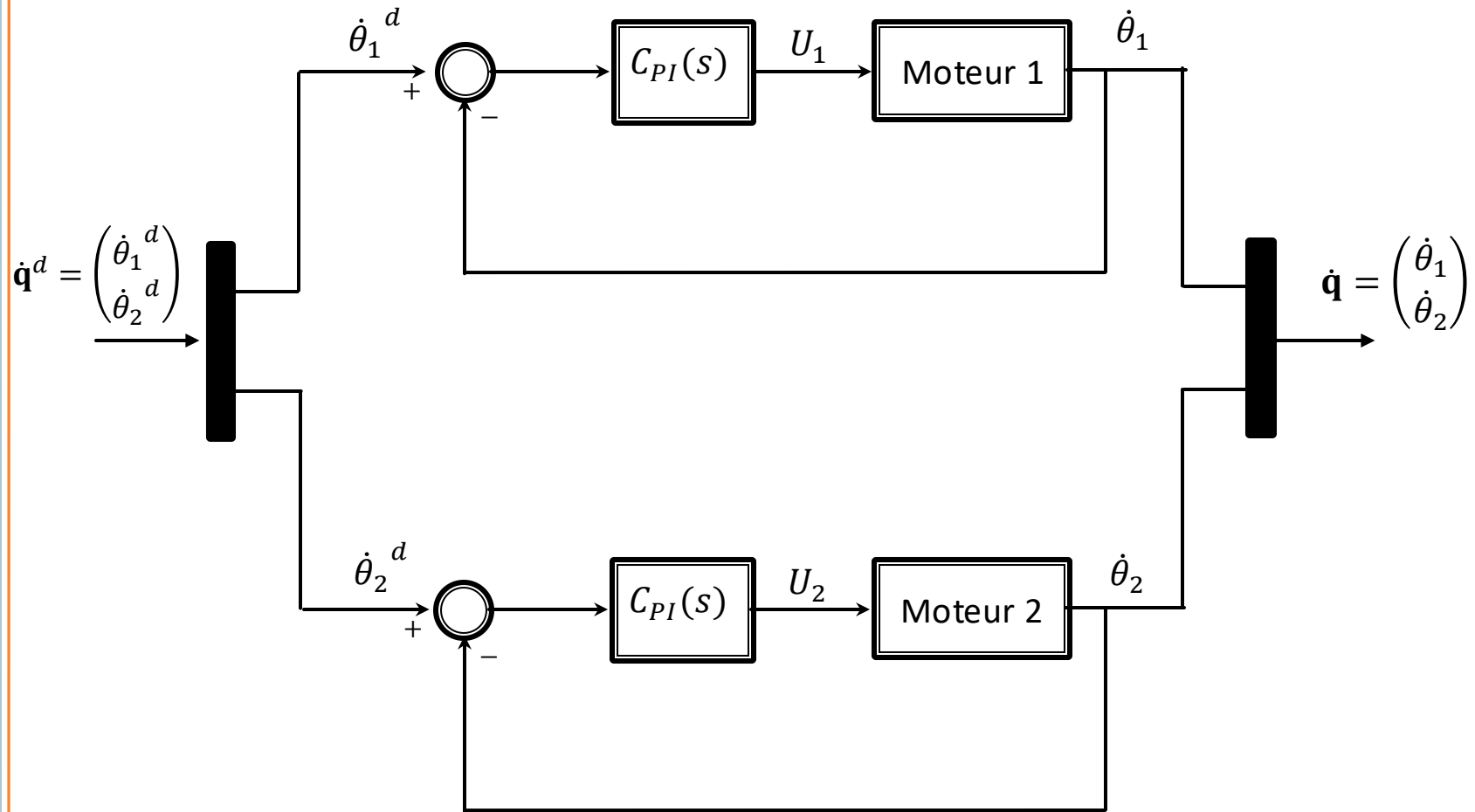


$$\mathbf{q} = \begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix}$$

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}$$

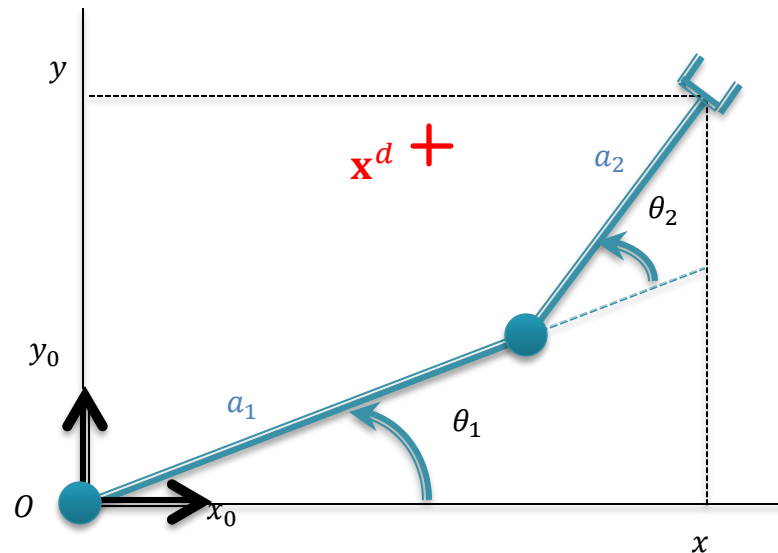
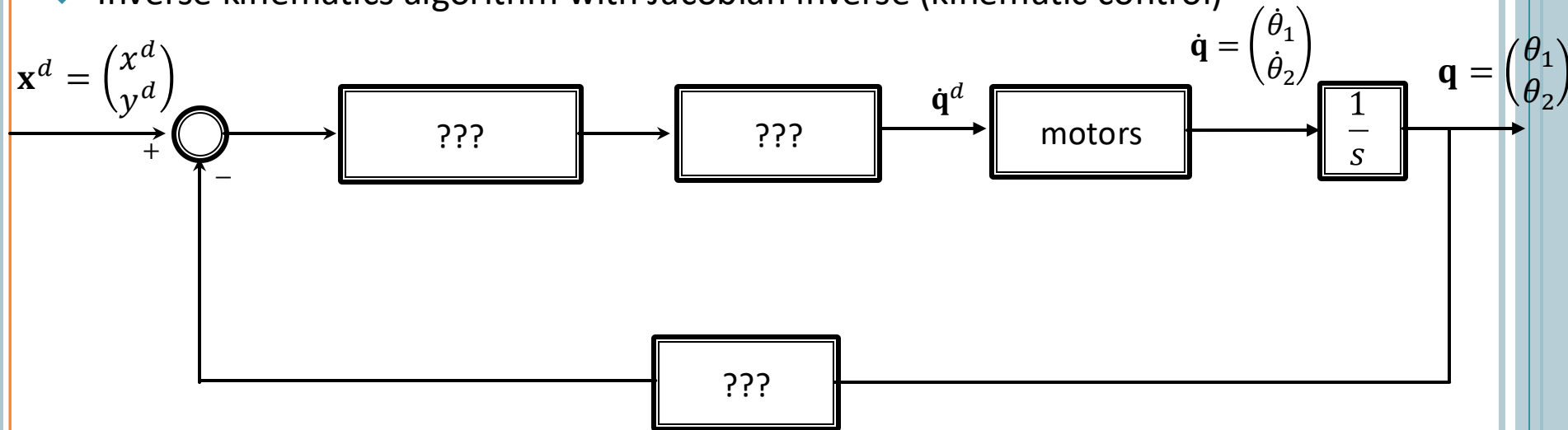
POSITION CONTROL

- ❖ Servomotors = motors controlled in velocity



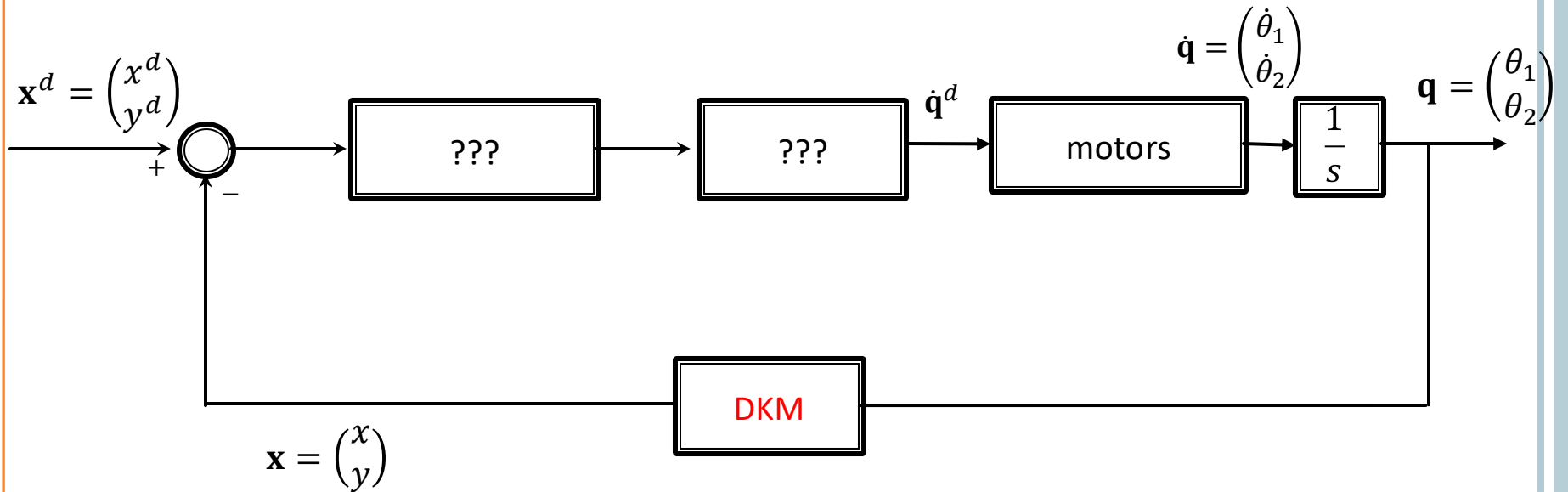
POSITION CONTROL

- ❖ Inverse kinematics algorithm with Jacobian inverse (kinematic control)



POSITION CONTROL

- ❖ Inverse kinematics algorithm with Jacobian inverse (kinematic control)



POSITION CONTROL

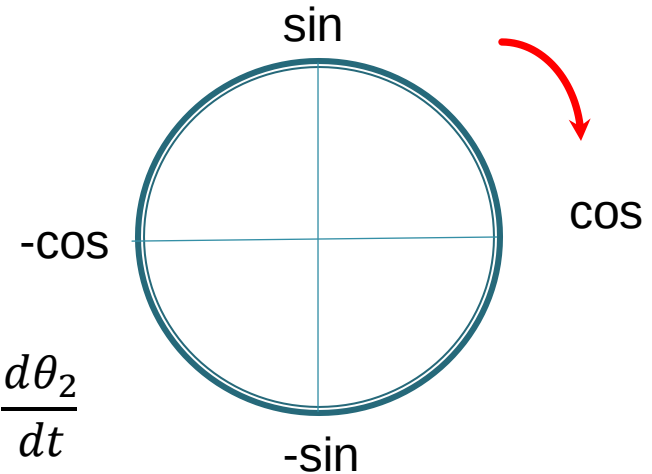
To establish the analytical Jacobian of the 2 DOF robot, it would be necessary to derive the direct kinematic model:

$$\text{DKM: } \begin{cases} x = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) \\ y = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2) \end{cases}$$

$$\dot{\mathbf{x}} = \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \end{pmatrix} = ??$$

POSITION CONTROL

$$\begin{cases} \frac{d \sin \theta}{d\theta} = \cos \theta \\ \frac{d \cos \theta}{d\theta} = -\sin \theta \end{cases}$$



$$\begin{aligned} \frac{dx}{dt} &= \frac{\partial x}{\partial \theta_1} \frac{d\theta_1}{dt} + \frac{\partial x}{\partial \theta_2} \frac{d\theta_2}{dt} \\ &= \frac{\partial x}{\partial \theta_1} \dot{\theta}_1 + \frac{\partial x}{\partial \theta_2} \dot{\theta}_2 \end{aligned}$$

For the record:

$$x = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2)$$

$$\begin{aligned} \frac{\partial x}{\partial \theta_1} &= -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) \\ \frac{\partial x}{\partial \theta_2} &= -a_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

POSITION CONTROL

$$\begin{cases} \frac{d \sin \theta}{d\theta} = \cos \theta \\ \frac{d \cos \theta}{d\theta} = -\sin \theta \end{cases}$$

$$\begin{aligned} \frac{dy}{dt} &= \frac{\partial y}{\partial \theta_1} \frac{d\theta_1}{dt} + \frac{\partial y}{\partial \theta_2} \frac{d\theta_2}{dt} \\ &= \frac{\partial y}{\partial \theta_1} \dot{\theta}_1 + \frac{\partial y}{\partial \theta_2} \dot{\theta}_2 \end{aligned}$$

For the record:

$$y = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

$$\begin{aligned} \frac{\partial y}{\partial \theta_1} &= a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) \\ \frac{\partial y}{\partial \theta_2} &= a_2 \cos(\theta_1 + \theta_2) \end{aligned}$$

POSITION CONTROL

$$\frac{dx}{dt} = \{-a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2)\}\dot{\theta}_1 + \{-a_2 \sin(\theta_1 + \theta_2)\}\dot{\theta}_2$$

$$\frac{dy}{dt} = \{a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2)\}\dot{\theta}_1 + \{a_2 \cos(\theta_1 + \theta_2)\}\dot{\theta}_2$$

$$\begin{aligned}\dot{\mathbf{x}} = \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} &= \begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \end{pmatrix} = \begin{pmatrix} -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) & -a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) & a_2 \cos(\theta_1 + \theta_2) \end{pmatrix} \begin{pmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{pmatrix} \\ &= \mathbf{J}_x(\mathbf{q})\dot{\mathbf{q}}\end{aligned}$$

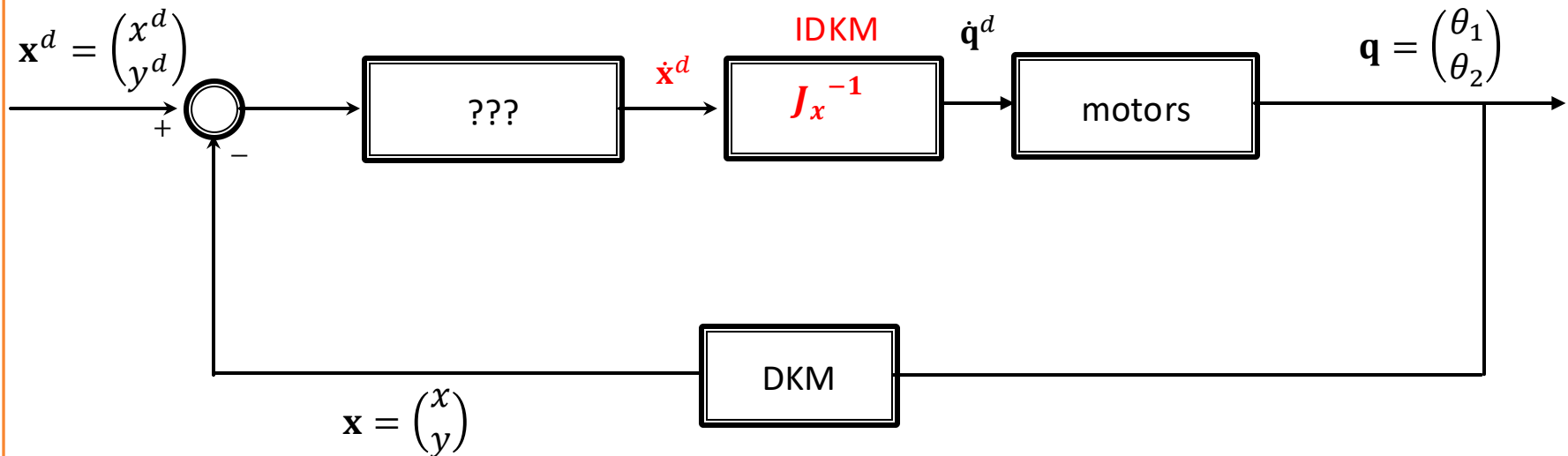
with:

$$\mathbf{J}_x(\mathbf{q}) = \begin{pmatrix} -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) & -a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) & a_2 \cos(\theta_1 + \theta_2) \end{pmatrix}$$

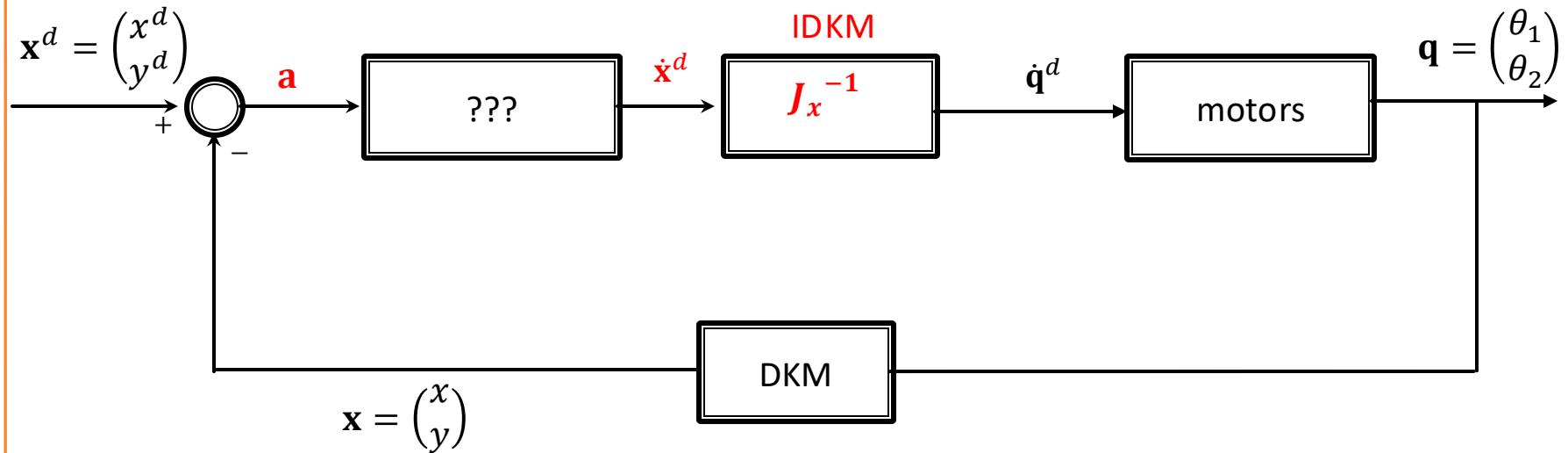
POSITION CONTROL

$$\dot{\mathbf{q}} = \mathbf{J}_x(\mathbf{q})^{-1} \dot{\mathbf{x}}$$

$\mathbf{J}_A(\mathbf{q})^{-1}$ is the inverse of the analytical Jacobian of the robot.



POSITION CONTROL

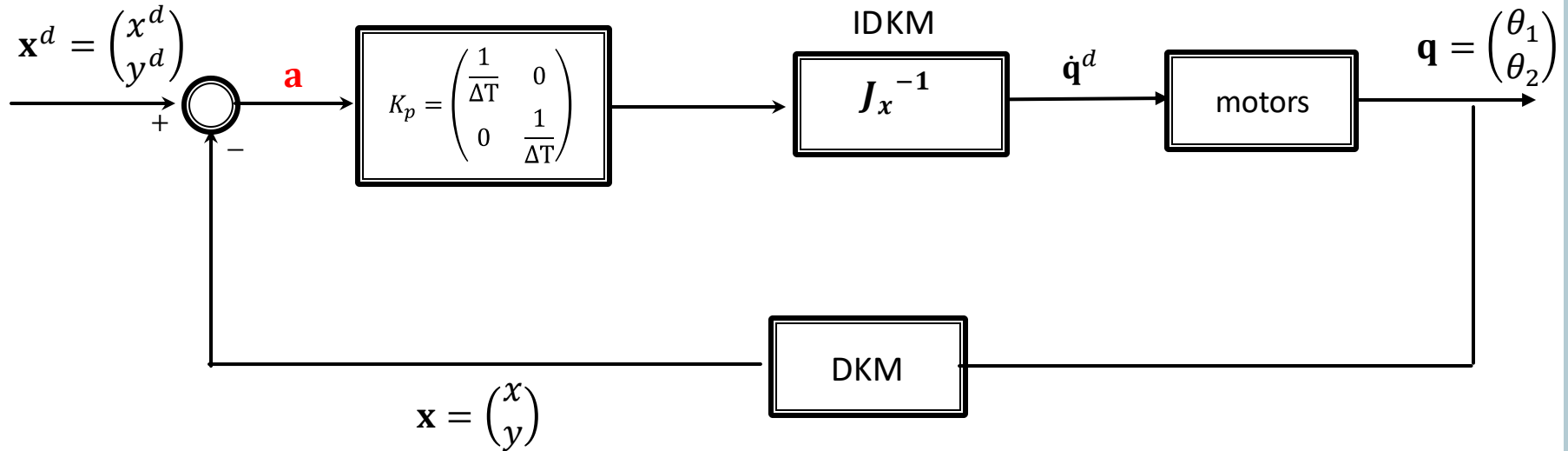


$$\mathbf{a} = \mathbf{x}^d - \mathbf{x}$$

$$= \begin{pmatrix} x^d \\ y^d \end{pmatrix} - \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x^d - x \\ y^d - y \end{pmatrix}$$

$$\dot{\mathbf{x}}^d = \begin{pmatrix} \dot{x}^d \\ \dot{y}^d \end{pmatrix}$$

POSITION CONTROL



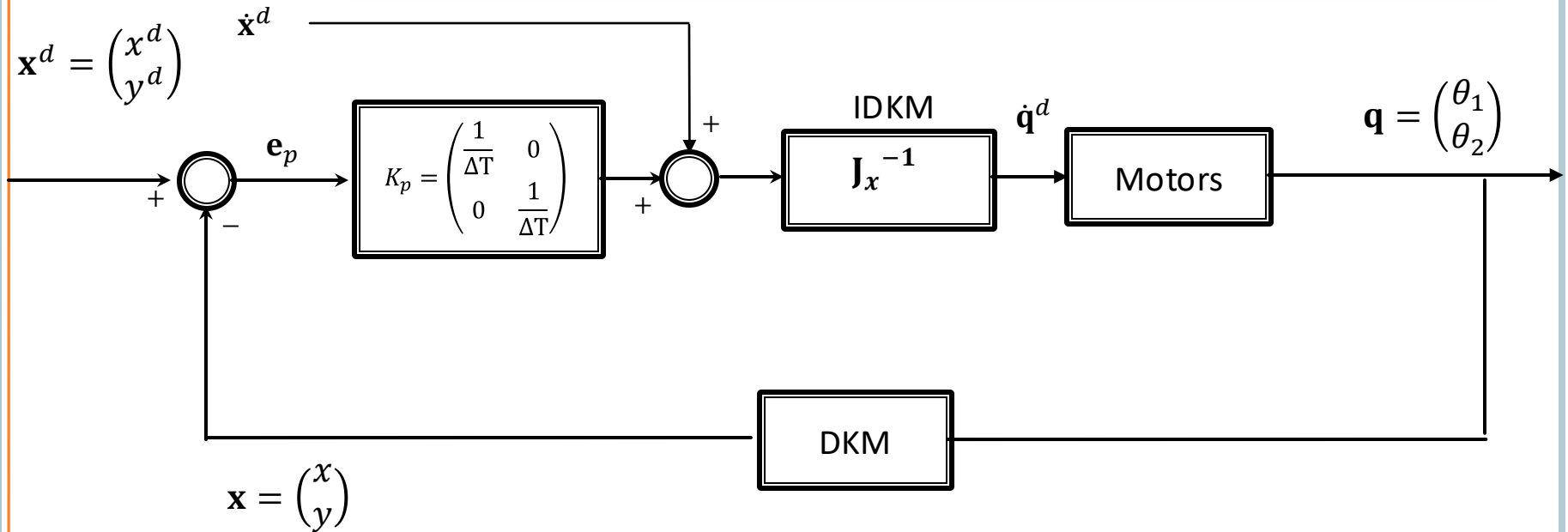
$$\mathbf{a} = \begin{pmatrix} x^d - x \\ y^d - y \end{pmatrix}$$

$$\dot{\mathbf{x}} = \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix}$$

Link between $\mathbf{a}$ and $\dot{\mathbf{x}}$??

$$\dot{\mathbf{x}} = \begin{pmatrix} \frac{1}{\Delta T} & 0 \\ 0 & \frac{1}{\Delta T} \end{pmatrix} \begin{pmatrix} x^d - x \\ y^d - y \end{pmatrix} = \begin{pmatrix} \frac{x^d - x}{\Delta T} \\ \frac{y^d - y}{\Delta T} \end{pmatrix}$$

POSITION CONTROL



kinematic control

POSITION CONTROL

Control law:

$$\dot{\mathbf{q}}^d = J_x^{-1}(\dot{\mathbf{x}}^d + K_p \mathbf{e}_p)$$

Robot model:

$$\dot{\mathbf{q}} = J_x^{-1} \dot{\mathbf{x}}$$

If the actuators are correctly controlled:

$$\dot{\mathbf{q}} = \dot{\mathbf{q}}^d$$

This leads to:

$$\dot{\mathbf{x}} = \dot{\mathbf{x}}^d + K_p \mathbf{e}_p$$



$$\dot{\mathbf{e}}_p + K_p \mathbf{e}_p = 0$$

If K_p is positive definite (usually diagonal) matrix, the system is *asymptotically stable*

POSITION CONTROL

- ❖ In the case of redundant manipulator, the control law $\dot{\mathbf{q}}^d = J_A^{-1}(\dot{\mathbf{x}}^d + K_p \mathbf{e}_p)$ can be generalized into:

$$\dot{\mathbf{q}}^d = J_A^+(\dot{\mathbf{x}}^d + K_p \mathbf{e}_p) + (I_n - J_A^+ J_A) \dot{\mathbf{q}}_0$$

- ❖ The matrix $(I_n - J_A^+ J_A)$ allows to generate internal motion described by $(I_n - J_A^+ J_A) \dot{\mathbf{q}}_0$ without changing the end effector position and orientation.

POSE CONTROL IN OPERATIONAL SPACE - POSITION ERROR

- ❖ The kinematic control scheme above presented uses the analytical Jacobian.
- ❖ For the position error, the expression is:

$$\mathbf{e}_p = \mathbf{x}^d - \mathbf{x}$$

$\mathbf{x}^d$ is the desired position of the end effector

$\mathbf{x}$ is the computed end effector position

- ❖ The first time derivative of the position error is:

$$\dot{\mathbf{e}}_p = \dot{\mathbf{x}}^d - \dot{\mathbf{x}}$$

- ❖ Orientation error depends on the particular representation of the end effector orientation, namely Euler angles, angle and axis, and unit quaternion

POSE CONTROL IN OPERATIONAL SPACE - ORIENTATION ERROR USING EULER ANGLES

❖ Let's define:

$\phi = \begin{pmatrix} \varphi \\ \theta \\ \psi \end{pmatrix}$ as the orientation of the end-effector

❖ The orientation error is:

$$\mathbf{e}_O = \phi^d - \phi(\mathbf{q})$$

ϕ^d represents the desired orientation

$\phi(\mathbf{q})$ is the computed set of Euler angles (orientation of the end effector)

❖ The first time derivative of the orientation error is:

$$\dot{\mathbf{e}}_O = \dot{\phi}^d - \dot{\phi}$$

POSE CONTROL IN OPERATIONAL SPACE - ORIENTATION ERROR USING EULER ANGLES

- ❖ Assuming that neither kinematic nor orientation representation singularities occur, the Jacobian inverse solution for nonredundant manipulator is:

$$\dot{\mathbf{q}} = J_A^{-1} \begin{bmatrix} \dot{\mathbf{x}}^d + \mathbf{K}_p \mathbf{e}_p \\ \dot{\boldsymbol{\phi}}^d + \mathbf{K}_o \mathbf{e}_o \end{bmatrix}$$

$\mathbf{K}_p$ and $\mathbf{K}_o$ are positive definite matrices

Difficulties:

- ❖ Computation of the orientation variables $\boldsymbol{\phi}$ from the joint variables $\mathbf{q}$ is not easy, except for simple cases
- ❖ The computation of the orientation variables $\boldsymbol{\phi}$ requires the computation of the rotation matrix ${}^0\mathbf{R}_n$.
- ❖ In fact, only the dependence of ${}^0\mathbf{R}_n$ on $\mathbf{q}$ is always known, but not that of $\boldsymbol{\phi}$ on $\mathbf{q}$
- ❖ The use of inverse functions atan2 involves a non-negligible complexity in the analytical Jacobian, and occurrence of representation singularities present another drawbacks

POSE CONTROL IN OPERATIONAL SPACE - ORIENTATION ERROR USING ANGLE AND AXIS

- ❖ From trajectory planning :

$$\mathbf{R}^d(t) = \mathbf{R}^i \text{rot}(\mathbf{u}, r(t)\vartheta)$$

- ❖ Let's define the vectors $\mathbf{n}^d$, $\mathbf{s}^d$ and $\mathbf{a}^d$ as the column of the matrix $\mathbf{R}^d(t)$:

$$\mathbf{R}^d = (\mathbf{n}^d \quad \mathbf{s}^d \quad \mathbf{a}^d)$$

- ❖ Let's define also define the vectors $\mathbf{n}^e$, $\mathbf{s}^e$ and $\mathbf{a}^e$ as the column of the matrix $\mathbf{R}^e$:

$$\mathbf{R}^e = (\mathbf{n}^e \quad \mathbf{s}^e \quad \mathbf{a}^e)$$

$\mathbf{R}^e$ represents the current orientation of the robot.

$$\mathbf{e}_O = \frac{1}{2} (\mathbf{n}^e \wedge \mathbf{n}^d + \mathbf{s}^e \wedge \mathbf{s}^d + \mathbf{a}^e \wedge \mathbf{a}^d)$$

POSE CONTROL IN OPERATIONAL SPACE - ORIENTATION ERROR USING ANGLE AND AXIS

❖ Differentiating $\mathbf{e}_O$ with respect to time gives:

$$\dot{\mathbf{e}}_O = \mathbf{L}^T \boldsymbol{\omega}_d - \mathbf{L} \boldsymbol{\omega}_e$$

With:

$$\mathbf{L} = \frac{-1}{2} \left(\mathbf{S}(\mathbf{n}^d) \mathbf{S}(\mathbf{n}^e) + \mathbf{S}(\mathbf{s}^d) \mathbf{S}(\mathbf{s}^e) + \mathbf{S}(\mathbf{a}^d) \mathbf{S}(\mathbf{a}^e) \right)$$

The matrix $\mathbf{S}$ is defined for a vector $\mathbf{n} = (n_x \quad n_y \quad n_z)^T$ so that :

$$\mathbf{S}(\mathbf{n}) = \begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}$$

The control law can be defined as:

$$\dot{\mathbf{q}}^d = \mathbf{J}^{-1}(\mathbf{q}) \begin{bmatrix} \dot{\mathbf{x}}^d + \mathbf{K}_p \mathbf{e}_p \\ \mathbf{L}^{-1}(\mathbf{L}^T \boldsymbol{\omega}_d + \mathbf{K}_O \mathbf{e}_O) \end{bmatrix}$$

With: $\boldsymbol{\omega}_d = \mathbf{R}^i \dot{\mathbf{r}}(t) \vartheta \mathbf{u}$

It is worth remarking that this control law works better than the one using Euler angles since it uses the geometric Jacobian instead of the analytical Jacobian, thus avoiding the occurrence of representation singularities

KINEMATIC SINGULARITIES

- ❖ The computation of the inverse or pseudo-inverse of the Jacobian is possible if and only if the Jacobian has full rank.
- ❖ In the case of a singularity (rank deficient), the system $\mathbb{V} = \mathbf{J} \dot{\mathbf{q}}$ contains linear dependent equations
- ❖ Solution (damped least square inverse):

$$\mathbf{J}^* = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + k^2 \mathbf{I})^{-1}$$

Where k is a damping factor that renders the inversion better conditioned from a numerical viewpoint.

KINEMATIC SINGULARITIES

- ❖ To determine the singularities of the robot, it is enough to solve the equation $\det(\mathbf{J}) = 0 \Rightarrow$ rank deficient of the matrix $\mathbf{J}$

Example: robot 2 DOF

$$\mathbf{J} = \mathbf{J}_A = \begin{pmatrix} -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) & -a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) & a_2 \cos(\theta_1 + \theta_2) \end{pmatrix}$$

Let's define:

- ❖ $S_1 = \sin \theta_1$
- ❖ $C_1 = \cos \theta_1$
- ❖ $S_{12} = \sin(\theta_1 + \theta_2)$
- ❖ $C_{12} = \cos(\theta_1 + \theta_2)$

$$\mathbf{J} = \begin{pmatrix} -a_1 S_1 - a_2 S_{12} & -a_2 S_{12} \\ a_1 C_1 + a_2 C_{12} & a_2 C_{12} \end{pmatrix}$$

KINEMATIC SINGULARITIES

$$\det(\mathbf{J}) = a_2 C_{12}(-a_1 S_1 - a_2 S_{12}) + a_2 S_{12}(a_1 C_1 + a_2 C_{12}) = 0$$

$$\Rightarrow \det(\mathbf{J}) = -a_1 a_2 (S_1 C_{12} - C_1 S_{12}) = 0$$

We have:

$$\sin(\alpha - \beta) = \sin\alpha \cos\beta - \cos\alpha \sin\beta$$

$$S_1 C_{12} - C_1 S_{12} = \sin\theta_1 \cos(\theta_1 + \theta_2) - \cos\theta_1 \sin(\theta_1 + \theta_2) = \sin(\theta_1 - (\theta_1 + \theta_2))$$

$$= \sin(-(\theta_2)) = -\sin(\theta_2) = -S_2$$

$$\Rightarrow \det(\mathbf{J}) = a_1 a_2 S_2 = 0$$

$$\Rightarrow S_2 = 0$$

$$\Rightarrow \theta_2 = 0 \text{ or } \theta_2 = \pi$$

❖ The robot is in a singular configuration if $\theta_2 = 0$ or $\theta_2 = \pi$